

ПОЛИМОРФИЗЪМ – АБСТРАКТНИ КЛАСОВЕ

В предния урок разгледахме статичен полиморфизъм чрез клас Calculator, който съдържа защитена променлива name и 2 конструктора – един празен и друг, който приема name. Конструкторите може да създадем с **ctor + Tab x 2**. Конструкторите са специални методи, които:

- 1) носят името на класа, в който се намират
- 2) не връщат тип (void, int, string...)

Задачата на конструкторите е да конструират новия обект като свържат стойностите, които е подал потребителя като input variables с валидираните в класа, а могат да извършат и допълнителни действия като sw.

Има родителски (базови) класове, чиято цел е да предоставят свойства и методи на наследниците, НО не могат да се създават обекти от родителите, а само от наследниците. В такива случаи, пред името на родителския клас се добавя **abstract**. По същия начин, в абстрактни родителски класове може да има както методи, с които сме свикнали, така и абстрактни методи – те нямат тяло, но името им се съхранява и дава на наследниците, които го презаписват чрез override.

Какво означава abstract? – идва от абстрахиране – отдалечаване. Това е една от концепциите на ООП – обектно-ориентираното програмиране. Когато имаме даден реален обект от заобикалящата среда, например маса, тя е толкова сложен обект с цялата съвкупност от качества – форма, размер, цвят, тегло, материал, плътност и т.н., че за да се вкара в компютърен вид, се използва абстрахиране, т.е. отдалечаване и опростяване (стилизиране в изкуството). Вземат се най-важните качества, които се необходими за описанието. Пример е абстрактното изкуство. Един онагледяващ цитат е:

„Някои художници изобразяват слънцето като жълто петно, но има други, които с помощта на своето изкуство и интелигентност, превръщат жълто петно в слънце.“ – Пабло Пикасо

There are painters who transform the sun to a yellow spot, but there are others who with the help of their art and their intelligence transform a yellow spot into the sun." - Pablo Picasso

Абстракционизъм, или **абстрактно изкуство** (на **латински**: *abstractio* – нещо отвлечено) е вид модернистично направление в изкуството, което се отличава с отказа си да изобразява предметите и събитията и с откъснатостта си от фигуративния подход към действителността. Една от целите му е да постигне **хармония** чрез съчетаване на **цветове** и геометрични форми, способни да предизвикат различни асоциации и **емоции**.

Възниква през 1910 – 1913 г. с разпояването на **кубизма** (който също използва геометрични фигури и пресичащи се плоскости), **експресионизма** и **футуризма**. Първите вълни абстрактни художници идват от течението на свободното рисуване, произлизащо от **експресионизма** – Василий Кандински и **Джаксън Полък** – и от близката до **кубизма** геометрична абстракция, на която се посвещават **Казимир Малевич**, **Пийт Мондриан**, **Франтишек Купка**.^[1]



Дефиниция [редактиране |

редактиране на кода]

Според уводната глава в книгата на **Мишел Рагон** „Произход на развитието на абстрактното изкуство“ (на **френски**: *Origines du développement de l'art abstrait*), цитираща предпазливата

дефиниция на абстрактното изкуство на **Мишел Съофор**:^[2]

Разликата между абстрактен и виртуален метод е, че **при виртуалния се дава тяло** в родителя, което наследникът може да използва наготово, а по желание да презапише своя версия с `override`, но не е задължително.

При **абстрактния метод**, само се запазва името и връщания тип, но **няма тяло** и затова **ЗАДЪЛЖИТЕЛНО** наследникът го презаписва с `override` и му задава тяло.

Абстрактните класове не могат да бъдат декларирани като `static` и не могат да бъдат забранявани със `sealed`.

Примерно използване на `abstract` в най-прост и нагледен вид.

Нека създадем абстрактен родителски клас `Animal`, без конструктор и демонстрираме в същото време разликите между 3-те вида методи:

- 1) обикновен, с тяло – с него сме работили най-много – `Sleep()`. Той стои в родителя, а наследникът го използва наготово. Той не е предназначен за презаписване – общ е, достъпен за наследниците. Ако се наложи като изключение някой от наследниците да използва същото име на метода `Sleep()`, но да промени тялото, той декларира, че е запознат и съгласен чрез `new`, който само временно ще скрие метода на родителя.
- 2) `virtual` – има тяло - той ПРЕДЛАГА действие, което наследникът може да презапише

с `override`, ако желае, а може да използва и наготово от родителя. Няма никакъв задължителен характер – не е нужно да бъде извикван.

3) `abstract` – в родителския абстрактен клас само се задава името на метода като абстрактно, без тяло, затова имплементирането му от наследниците е **ЗАДЪЛЖИТЕЛНО** – те са длъжни да му дадат код (тяло). Не може един метод да стои само с име. Както в `Main()` извикваме само името на методите, които си подреждахме нагледно и логически, но кодът бе отделен извън него, така родителския клас само дава името, но кодът се изгражда в наследниците.

```
abstract class Animal // РОДИТЕЛ няма да създава new Animal();
{
    2 references
    public string Name { get; set; }
    1 reference
    1) public void Sleep() взема се наготово без промени
    {
        Console.WriteLine("The animal is sleeping.");
    }
    2 references
    2) public virtual void Eat() тяло -> override
    {
        Console.WriteLine("The animal is eating.");
    }
    2 references
    3) public abstract void Move(); само име -> override,
}
```

И един наследник за куче:

```
class Dog : Animal // наследник 1
{
    2 references
    public override void Eat() <- virtual
    {
        Console.WriteLine($"{Name} is eating."); // В Main() .Name без ctor
    }
    2 references
    public override void Move() <- abstract
    {
        Console.WriteLine($"The dog is moving."); НОВО ТЯЛО
    }
}
```

Особени моменти:

Още при влизането в абстрактен клас, IntelliSense ни предлага абстрактно публично свойство, което за по-голяма защита е само с `get` (read-only). То ще бъде **ЗАДЪЛЖИТЕЛНО** за използване и презаписване, което излишно ще усложни нашия код на този етап, затова ще продължим да си използваме автоматичните свойства:

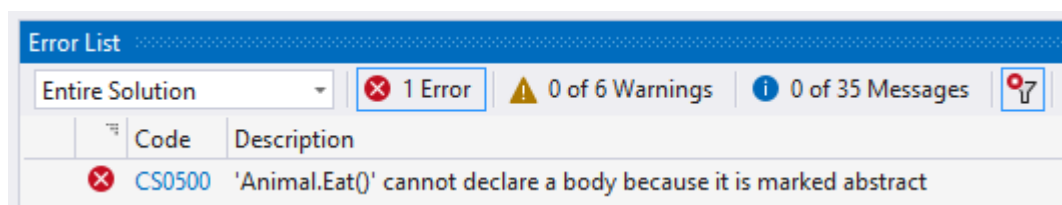
```
abstract class Animal
{
    public abstract string Name { get; }
}
```

Какво би се случило, ако по инерция напишем код в родителския абстрактен метод?

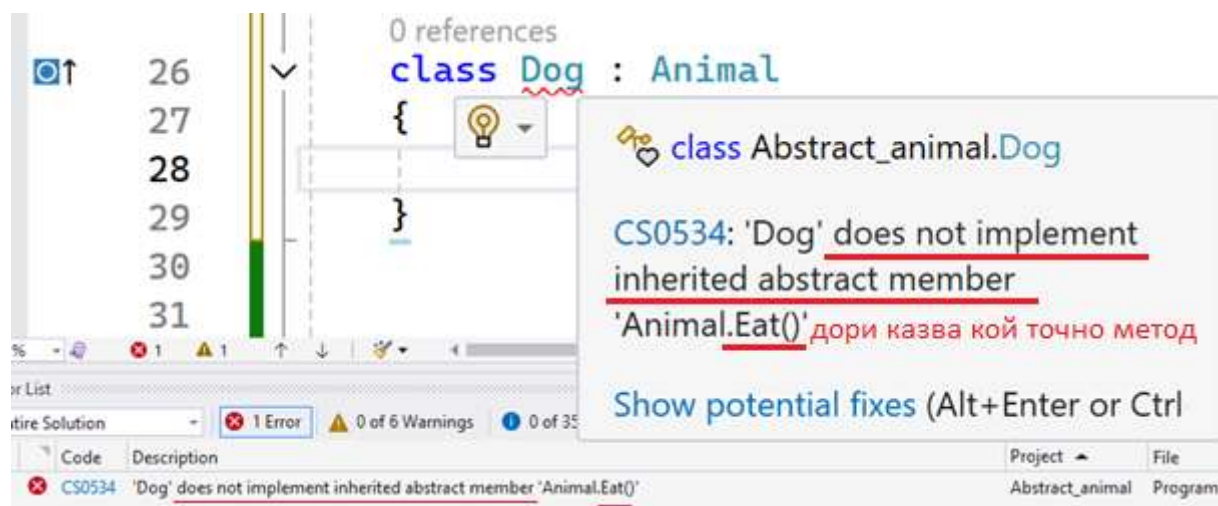
```
0 references
public abstract void Eat()
{
    Console.WriteLine($
}
```

void Animal.Eat()
CS0500: 'Animal.Eat()' cannot declare a body because it is marked abstract

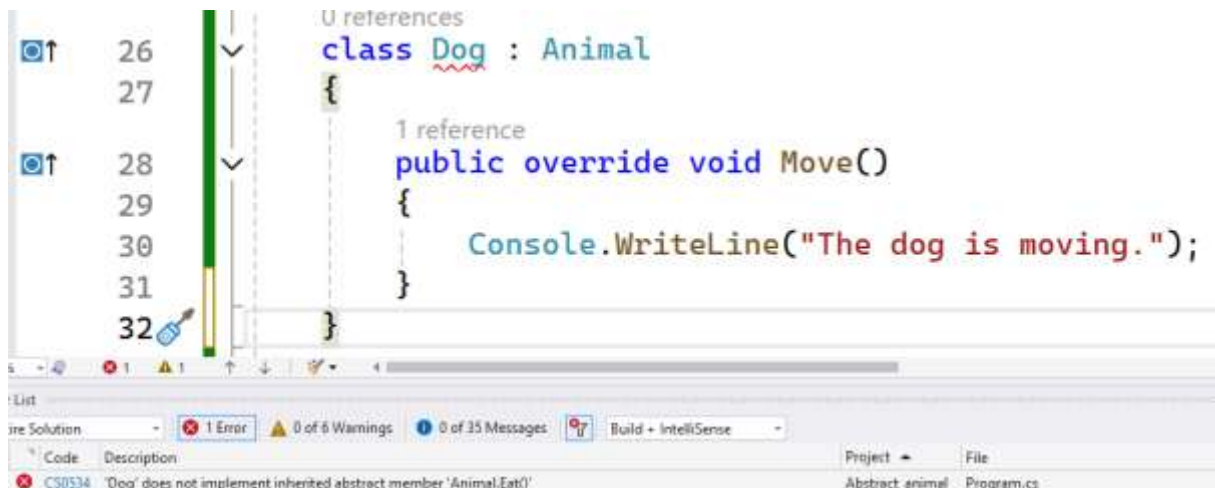
Грешката алармира, че методът не може да декларира тяло, защото е маркиран като абстрактен.



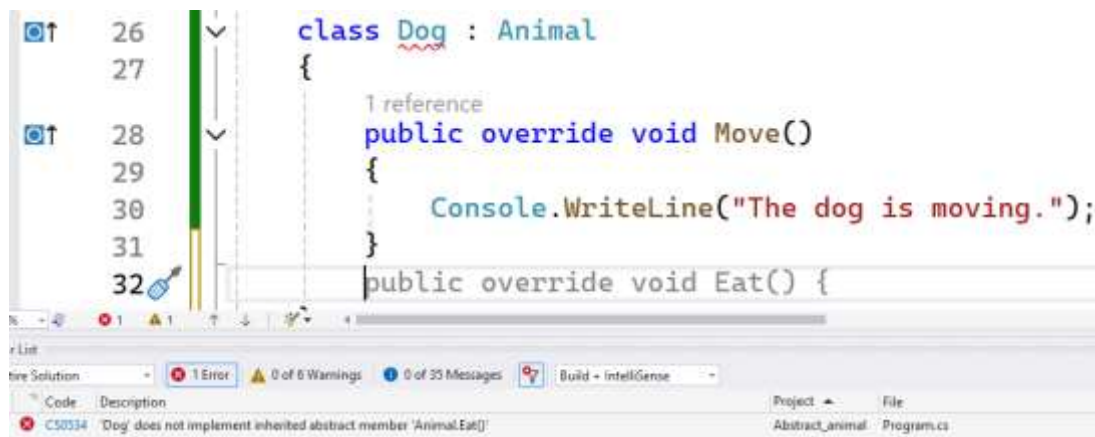
Когато създадем наследник на абстрактен родителски клас, не бива да ни смущава, че стои с червена зигзагообразна индикация за грешка. Ако дадем курсора върху нея, подсказката е, че като наследникът още не е имплементирал задължителния абстрактен метод и дори казва кой точно.



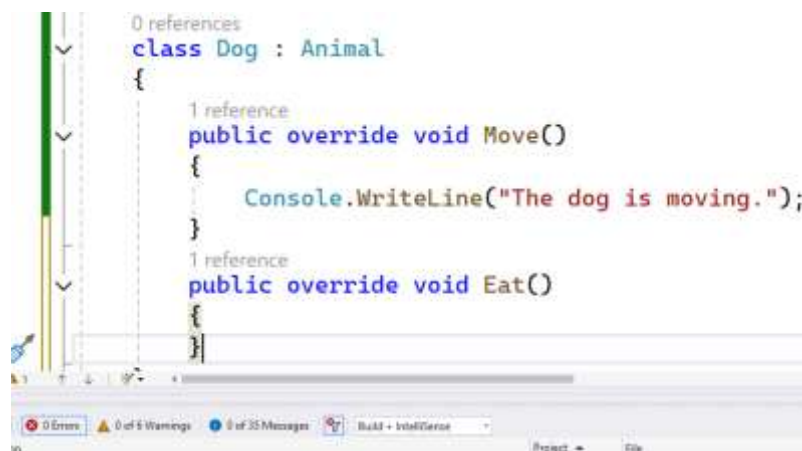
Можем да продължим да пишем правилно код – имплементираме други методи, но докато не имплементираме абстрактния, който е задължителен, грешката няма да се махне.



След това, IntelliSense се опитва да ни го напише той – директно. Толкова е задължителен, че не просто ни го подсказва, а направо натрапва с `override`, да не вземем да го забравим:

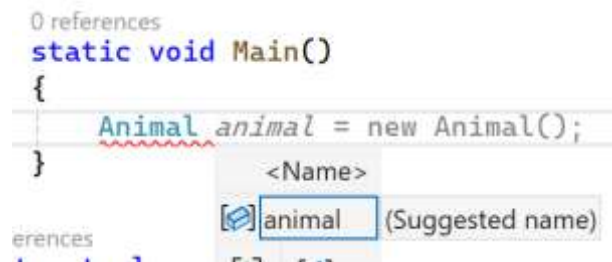


В същия миг, в който го потвърдим с Tab или дясна стрелка (зависи от настройките), грешката моментално изчезва:

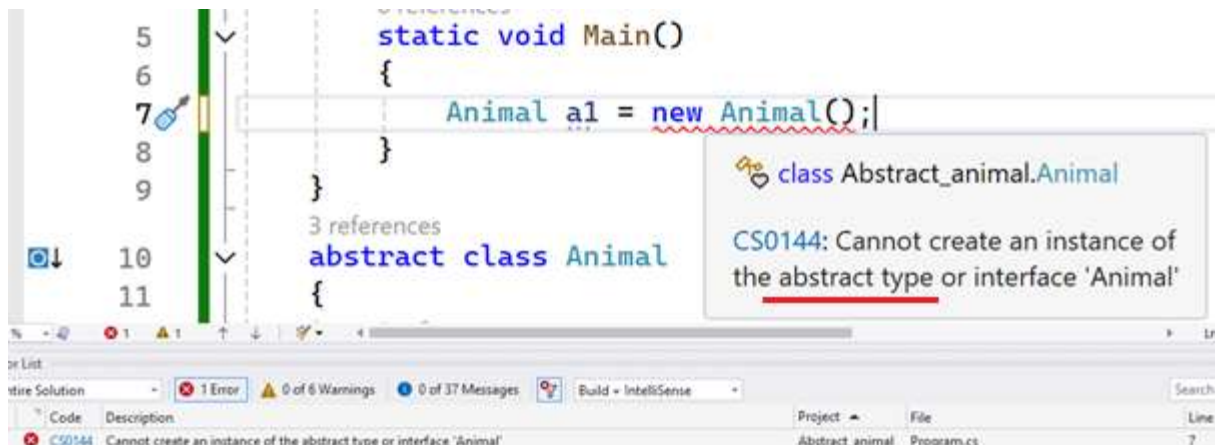


Забележете, че още дори няма код между къдравите скоби, но компилаторът вече е спокоен.

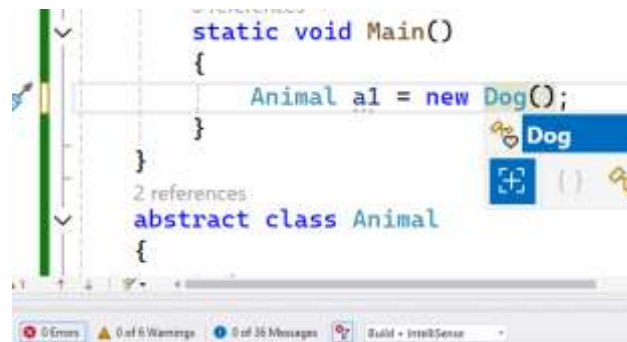
Когато дойде време да създаваме нови обекти в Main(), IntelliSense се опитва да ни подсказва отново, но нека видим какво ще стане, ако се подведем и потвърдим:



Ние създадохме нов абстрактен родител. Нека дадем курсора върху подчертаването:



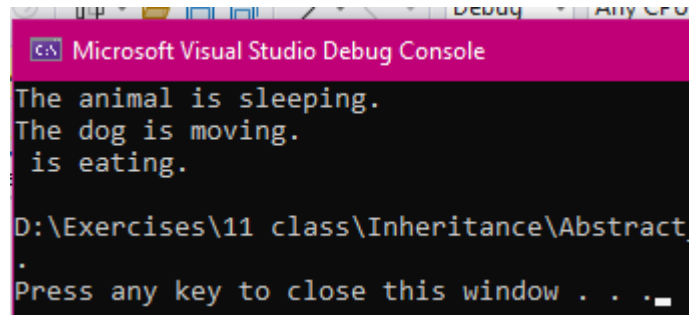
Не може да създаде обект (инстанция на класа), защото е абстрактен тип. Можем обаче да създадем нов обект на наследника:



Той е напълно достатъчен за да добием представа. За него може да извикаме всички методи, които се предоставят и от родителя, и от наследника. По същия начин, както досега, ако даден метод не присъства в наследника, но родителят го има, наследникът го взема автоматично (той наследява всичко от родителя).

```
static void Main()
{
    Animal a1 = new Dog();
    a1.Sleep(); // от родителя
    a1.Move();
    a1.Eat();
}
```

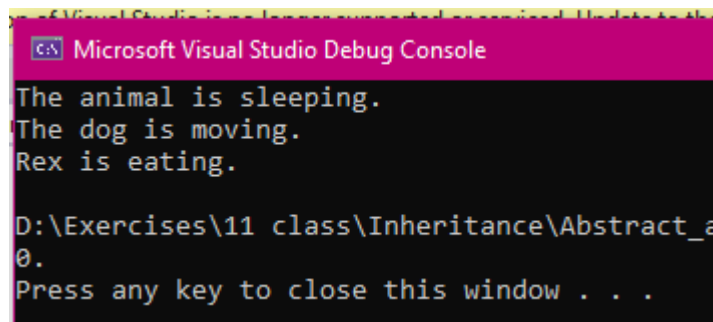
В случая умишлено не съм задала Name, за да видим изхода:



```
Microsoft Visual Studio Debug Console
The animal is sleeping.
The dog is moving.
is eating.
D:\Exercises\11 class\Inheritance\Abstract_...
Press any key to close this window . . .
```

Но ние имаме достъп до .Name, понеже е public и можем да го зададем и използваме дори без конструктор:

```
static void Main()
{
    Animal a1 = new Dog();
    a1.Sleep(); // от родителя
    a1.Move();
    a1.Name = "Rex";
    a1.Eat();
}
```



```
Microsoft Visual Studio Debug Console
The animal is sleeping.
The dog is moving.
Rex is eating.
D:\Exercises\11 class\Inheritance\Abstract_a...
Press any key to close this window . . .
```

Целият код:

```
internal class Program
{
    static void Main()
    {
        Animal a1 = new Dog();

        a1.Sleep(); // достъпно директно от родителя
```

```

        a1.Move();
        a1.Name = "Rex"; // без конструктор
        a1.Eat();
    }
}
abstract class Animal // РОДИТЕЛ
{
    public string Name { get; set; }
    public void Sleep()
    {
        Console.WriteLine("The animal is sleeping.");
    }

    public virtual void Move()
    {
        Console.WriteLine("The animal is moving");
    }

    public abstract void Eat(); // понеже е абстрактен, има само име без тяло
}
class Dog : Animal // наследник1
{
    public override void Move()
    {
        Console.WriteLine("The dog is moving.");
    }
    public override void Eat()
    {
        Console.WriteLine($"{Name} is eating.");
    }
}
}

```